PHASE 1 · OPERATIONAL FOUNDATION · VERSION 2

Codex Prompt & Execution Spec

Locked Phase 1 authority document — not a one-shot mega-prompt.

ONE STEP Per Codex Run	STOP & REPORT At Every Gate	NO SILENT Migrations

"The app becomes reliable first. Then it becomes intelligent."

Document Type	Locked Phase 1 Authority Document + Sequenced Codex Prompts
Version	v2 — May 2026 · Four review fixes applied (see change log)
Hierarchy	Phase 1 Spec wins over Grand Master Plan on all conflicts
Usage Rule	Run ONE step per Codex pass. Stop and report after each gate. Job Zero first.
Save As	/docs/phase-1-executable-spec.md (or attach as companion PDF)
Company	Cultivated Shelter LLC · CCB# 235357 · Portland, OR · shelterprep.io

TABLE OF CONTENTS

Phase 1 Codex Prompt v2

> **v2 CHANGES: (1) Phased execution protocol added · (2) files schema corrected — signed URLs generated on demand, not stored · (3) Steps 8 & 9 split into diagnosis vs. execution · (4) No silent DB migrations — stop and report before applying. Full change log in Appendix.**

PART 1

Doctrine, Hierarchy & Execution Protocol

Document Authority

Three documents govern the build. When they conflict, this hierarchy resolves it:

Grand Master Plan	Vision only. Not a build spec. Codex reads it for context, not instructions.
Reconciliation Map	Conflict-resolution rules. Defines which document wins when specs diverge.
Phase 1 Execution Spec	What Codex builds now. This document. Wins all conflicts with the Master Plan.

Execution Protocol — Read First

> **This document defines the FULL Phase 1 execution sequence. Codex must NOT implement all steps in one pass. Codex executes ONE step at a time and STOPS after each gate. The first run is Job Zero only.**

Use this PDF as the locked authority document — not as a single mega-prompt. Pasting the whole document into Codex and asking it to “build all of this” invites one giant risky refactor: compressing code, creating schema, adding storage, adding RLS, and scaffolding agents all at once. Instead, hand Codex one step per run, with the stop-and-report gate below. The ready-to-paste prompt for each step is in Part 11.

The Stop-and-Report Template (applies to every step)

```
After completing the step, STOP and report:
1. Files changed (created / modified / moved)
2. What was extracted or added
3. Acceptance checks run and their results
4. npm run build result
5. npx tsc --noEmit result
6. Risks, follow-ups, or anything that needs a human decision
Do not begin the next step until explicitly told to proceed.
```

Database Safety Rule

> **Codex may CREATE migration files, but must NOT assume migrations are applied. After generating any schema, RLS, or storage-policy migration, Codex stops and provides: migration summary · tables created · policies created · a reversible down-migration · risk notes · a manual test checklist. A human applies migrations.**

The Immutable Build Constraint

DO NOT add features. DO NOT add agents beyond the six. DO NOT scaffold Phase 2+ structures. The app becomes reliable first — then it becomes intelligent.

Operating Principle

Manual first → Assisted second → Automated later

The browser must not decide what is verified.

review_required is computed server-side. Confidence is triage only.

Anything affecting pricing, scope, safety, legality, or contractor routing must be human reviewed.

PART 2

Job Zero — Codex Prompt

This is the first prompt to run — before any other build work. Paste it directly into Codex. It implements Job Zero ONLY and stops.

```
Read /AGENTS.md, /docs/README.md, and /docs/phase-1-executable-spec.md.
Use ShelterPrep_Phase1_CodexPrompt_v2.pdf as the Phase 1 authority document.
Implement ONLY Job Zero from Part 2.
# GOAL
Compress and stabilize the current app so future phases can be built safely.
# RULES
- Do not implement Step 3 or beyond.
- Do not touch Supabase schema.
- Do not add features.
- Do not add agents.
- Do not change any Supabase calls' behavior.
- When extracting the data layer into src/lib/db/, preserve exact
existing query behavior.
- Move code. Do not rewrite it.
# REQUIRED WORK
1. Reduce App.tsx to a thin shell under 300 lines.
2. Extract screens into src/pages/.
3. Extract reusable UI into src/components/.
4. Extract shared styles/theme into src/theme/.
5. Extract types into src/types/.
6. Move existing data/Supabase/local-storage calls into src/lib/db/
without behavior changes.
7. Move placeholder agent/intelligence logic into src/agents/
without enabling new agents.
8. Preserve the current working intake/admin flow.
# STOP AND REPORT
1. Files changed
2. What was extracted
3. Whether App.tsx is under 300 lines
4. npm run build result
5. npx tsc --noEmit result
6. Any risks or follow-up issues
Commit message: Job Zero stabilization.
```

Target Directory Structure After Job Zero

```
src/
pages/ # Extracted screen components
components/ # Reusable UI elements
theme/ # Shared styles, colors, typography
lib/
db/ # All Supabase/data calls (behavior unchanged)
agents/ # Placeholder agent logic (not yet enabled)
types/ # Shared TypeScript types
App.tsx # Thin shell, under 300 lines
```

STOP CONDITION: No next step may begin until npm run build and npx tsc --noEmit both pass, and you are explicitly told to proceed.

PART 3

Acceptance Criteria

Each gate must be satisfied before Codex advances. These are blockers, not aspirations.

Job Zero Gate	npm run build passes · npx tsc --noEmit passes · App opens · Intake/admin flow works · No giant App.tsx · No broken imports · No new secrets
Schema Gate	Migration files generated AND reviewed by a human before apply · All canonical tables exist · App reads from tables, not local state · No 'leads' concept
File Storage Gate	Files land in Supabase Storage private bucket · Permanent file records in DB · Signed URLs generated on demand server-side · file_access_events logged
Auth / RLS Gate	RLS migration reviewed before apply · Roles enforced: admin, agent/PM, contractor, viewer · RLS prevents cross-tenant access · No hardcoded admin PIN in production
Trust Spine Gate	workflow_events immutable · review_required computed server-side · Browser cannot self-authorize verification · Audit log complete
Phase 1 Loop Gate	Full loop runs end to end · 50+ founder-verified workflows in memory · Majority of cohort agents return unprompted

PART 4

Build Sequence — Steps 1–10

Canonical order. Do not reorder. Do not skip. Each step is a prerequisite for the next, and each runs as a separate Codex pass.

Step 1 · Compress Code (Job Zero)

· Reduce App.tsx to under 300 lines.

· Extract: src/pages/, src/components/, src/theme/, src/lib/db/, src/agents/, src/types/

· Move code. Do not rewrite behavior. Do not change Supabase calls.

· Do not add features. Do not add agents.

Step 2 · Make the Build Pass

· Run: npm install · npm run build · npx tsc --noEmit

· Fix only build/type/import/runtime issues created or exposed by Job Zero.

· Do not add features. Do not touch schema. Do not add agents.

· App opens. Intake/admin flow works. No blank screen. No broken imports.

Step 3 · Replace Local State with Supabase Tables

· Generate migration files for canonical tables: properties, work_requests, files, repair_items, contractor_assignments, review_events, workflow_events

· STOP after generating migrations. Provide summary, down-migration, risk notes, and a manual test checklist. A human applies the migration.

· App stops thinking in 'leads'. Starts thinking in: Property → Work Request → Files → Repair Items → Review → Report

Step 4 · Build Real File / Photo Storage

· Supabase Storage with private buckets only.

· Permanent file records written to the files table on upload.

· Signed URLs generated ON DEMAND server-side — never stored as a column.

· file_access_events logged on every retrieval (with signed-URL lifecycle).

· Success: submit property → upload photo/PDF → file lands in Storage → admin opens file via signed URL → access event logged.

· DO NOT add AI before this works.

Step 5 · Add Auth, Roles, and RLS

· No hardcoded admin PIN long-term.

· Roles: admin · agent/PM · contractor · viewer/seller

· Admins manage all. Agents/PMs see own properties.

· Contractors see assigned work only. Sellers/viewers see report-level only.

· Anonymous users see no protected workflow data.

· Generate RLS migration, STOP, and report before any human applies it.

Step 6 · Build the Server-Side Trust Spine

· Server-side enforcement: workflow transitions, review_required, human_verified,

audit logs, immutable workflow_events, failure records, idempotency.

· The browser must not decide what is verified.

· review_required computed server-side only. Confidence is triage, never authorization.

Step 7 · Build the Work-Request / File Loop UI

· Primary statuses: New · In Progress · Ready · Done

· Secondary flags: Needs Info · Needs Review · Contractor Assigned ·

Estimate Drafted · Human Approved · Blocked

· Loop: Start Property → New Work Request → Upload → Admin Review →

Status Change → Evidence View

· Keep the app calm. Not heavy construction software.

Step 8 · Manual Repair Item Review — WHAT IS WRONG

· Tables: repair_items, missing_information, admin_corrections, review_events

· Manual approve / edit / reject flow.

· Founder must be able to create, correct, and approve repair items by hand.

· This step captures the diagnosis — the defects and gaps found.

Step 9 · Job Execution Scope Structures — HOW WORK GETS DONE

· Tables: job_execution_steps, scope_notes, estimate_review_owner

· Plus: materials / tools / equipment, access notes, safety notes,

cleanup / disposal.

· This step captures execution — how the diagnosed work actually gets performed.

· No AI automation until these structures are battle-tested by humans.

Step 10 · Add Six Phase 1 Agents Only — One at a Time

· Agents: Intake · Inspector · Gap Detection · Scope Drafter · Report · UX/Presentation

· Estimate and contractor routing stay manual in Phase 1.

· DO NOT scaffold the full future agent taxonomy.

· DO NOT add: Seller Prep, predictive maintenance, contractor portal,

pricing automation, source research, or the Observer/Analyst agent.

· Each agent added one at a time. Each must reduce proven friction.

> **Simple order to remember:**
>
> **1 Compress → 2 Build Pass → 3 Tables → 4 Storage → 5 Auth/RLS**
>
> **6 Trust Spine → 7 File Loop → 8 Diagnosis (what's wrong) → 9 Execution (how) → 10 Six Agents**

PART 5

Agent Roster — Six Live, Four Deferred

An agent earns its place by reducing proven friction. Phase 1 runs six agents and defers four. Deferred functions are performed manually by the founder — that manual work becomes the training signal deferred agents later learn from.

AGENT	STATUS	RATIONALE
Intake	**LIVE**	Unstructured uploads are immediate friction from day one. Classifying them is pure value.
Inspector	**LIVE · founder-corrected**	AI drafts repair items; founder corrects every one. This is the training environment.
Gap Detection	**LIVE**	Cheap, high-value, prevents the rework that destroys contractor trust early.
UX / Presentation	**LIVE**	Calm onboarding IS the adoption strategy for 30 agents. Live from day one.
Scope Drafter	**LIVE · founder-owned**	Drafts scope; founder rewrites freely. Where scope-quality training data forms.
Report	**LIVE · founder-owned**	The deliverable that makes agents look like pros. Founder edits every one.
Estimate	**DEFERRED → Phase 2**	Needs pricing memory to be useful. None exists yet. Founder estimates manually; that seeds the memory.
Routing	**DEFERRED → Phase 2**	Founder knows all 15–20 contractors personally. Manual matching beats an algorithm at this scale.
Memory	**MANUAL · light**	Records staged from the start; founder commits them directly. No sophisticated agent needed yet.
Observer / Analyst	**DEFERRED → Phase 2–3**	Nothing to observe. Low volume = noise, not patterns. Building it now would mislead.

The Founder Is the Estimate Engine, the Router, and the Memory-Committer

This is not a gap — it is the design. Every manual estimate made with GC judgment, every contractor matched by hand, every record committed, is the verified signal the deferred agents will train on. Phase 1 builds 6 agents, not 10 — a meaningfully smaller engineering lift for the first Head of Engineering hire.

PART 6

Schema & Data Architecture

The app must stop thinking in 'leads' and adopt the property-centered model below.

Canonical Tables

```
# Core workflow tables
properties -- id, address, status, owner_id, created_at
work_requests -- id, property_id, type, status, review_required
files -- id, work_request_id, storage_bucket, storage_path,
original_name, mime_type, size_bytes, uploaded_by,
created_at [PERMANENT — no signed URL column]
repair_items -- id, work_request_id, category, description, status
contractor_assignments -- id, repair_item_id, contractor_id, assigned_by
review_events -- id, work_request_id, reviewer_id, outcome, notes
workflow_events -- id, entity_id, event_type, actor, payload [IMMUTABLE]
# File access tracking — signed URLs are temporary, generated on demand
file_access_events -- id, file_id, accessor_id, signed_url_created_at,
expires_at, accessed_at
# Supporting tables
contractors -- id, name, ccb_license, bond_status, trade_type
agents -- id, user_id, role, agency
```

v2 fix: signed_url_expiry is NOT a column on files. A signed URL is ephemeral — it is generated server-side when access is needed, and its lifecycle (created/expires/accessed) is recorded in file_access_events. The file record itself stays permanent.

Property-Centered Mental Model

```
Property
■■■ Work Request (one per coordination event)
■■■ Files (photos, PDFs, notes) — permanent records
■■■ Repair Items (AI-drafted, human-verified)
■ ■■■ Contractor Assignments
■■■ Review Events (human gates)
■■■ Workflow Events (immutable audit trail)
```

Workflow Verification States

Every important output carries a verification status. These flow one direction — the browser cannot downgrade them.

AI Draft	Raw output. Not shown to clients or contractors.
Needs Review	Flagged for human inspection. Loop blocked until resolved.
Human Reviewed	Admin has seen and accepted. Can be sent to agents.
Human Verified	Admin has positively confirmed against field reality.
Contractor Verified	Contractor has confirmed scope against site conditions.
Completed / Actual	Outcome recorded. Enters operational memory.

PART 7

File Storage & RLS

The most important reliability milestone before AI. Current MVP weakness: file names show but real files are not retrievable.

```
# Required implementation
Supabase Storage -- private buckets only
files table -- permanent record per upload
Signed URLs -- generated ON DEMAND, server-side, time-limited
NEVER stored as a column on files
file_access_events -- log signed-URL lifecycle + every retrieval
Admin preview / download -- via signed URL, never direct bucket URL
# Success condition
submit property
↓ upload photo or PDF
↓ file lands in Supabase Storage private bucket
↓ permanent file record written to files table
↓ admin requests access → signed URL generated on demand
↓ file_access_event logged (created_at, expires_at, accessed_at)
# DO NOT add AI before this works.
```

RLS — Row Level Security Rules

admin	SELECT, INSERT, UPDATE, DELETE on all tables
agent / PM	SELECT own properties and work_requests · INSERT new work_requests
contractor	SELECT assigned work_requests and repair_items only
viewer / seller	SELECT report-level data only · No workflow internals
anonymous	No access to any protected workflow data

Contractors are scoped participants, not open marketplace browsers. RLS must enforce this — not just the UI. Generate the RLS migration, then STOP and report before a human applies it.

PART 8

Server-Side Trust Spine

The review-gate doctrine: the browser must not decide what is verified. Server-side enforcement is not optional.

```
# Server-side must enforce:
workflow_transitions -- state machine, not UI button logic
review_required -- computed server-side from scope/risk rules
human_verified -- set only by authenticated admin action
audit_logs -- append-only, never editable
workflow_events -- immutable insert-only table
failure_records -- logged on every agent failure
idempotency -- duplicate submissions create no duplicate records
# Review gate doctrine:
confidence scores -- triage only. Never authorization.
review_required -- triggers on: pricing, scope, safety, legality,
contractor routing
human gate -- nothing exits review_required without an
authenticated admin action
```

What Requires Human Review (Never AI-Only)

Pricing and estimate finalization

Scope changes affecting safety or structural systems

Contractor routing decisions

Legal or permit-required work classification

Any output marked review_required by server-side logic

Conversion of AI Draft to Human Reviewed status

PART 9

Work-Request / File Loop UI

Once the backend is reliable, build the operational loop. Keep the app calm — not heavy construction software.

Primary Visible Statuses

```
New -- Work request created, awaiting upload
In Progress -- Files uploaded, AI organizing or human reviewing
Ready -- Human-reviewed scope ready for contractor
Done -- Work completed and outcome recorded
```

Secondary Flags (Admin / Internal Only)

```
Needs Info -- Gap Detection flagged missing information
Needs Review -- review_required = true, awaiting admin action
Contractor Assigned -- Routing complete, awaiting contractor response
Estimate Drafted -- Scope Drafter output ready for human review
Human Approved -- Admin has verified and approved output
Blocked -- Dependency or readiness failure — action required
```

The Operational Loop Screen Flow

```
Start Property
↓ New Work Request
↓ Upload Photos / Documents
↓ AI Organizes (Intake + Inspector agents)
↓ Gap Detection flags missing info
↓ Admin Review (human gate — required)
↓ Scope Drafter generates contractor-ready scope
↓ Admin Approves Scope (human gate — required)
↓ Contractor Receives Structured Scope
↓ Track Progress
↓ Report Agent generates seller/client summary
↓ Outcome saved to operational memory
```

Adoption Psychology — Design for the Agent's Real Fear

An agent's anxiety: 'Will this make me look disorganized in front of my seller?'

The lever is not ease — it is making them look like a pro.

Every output an agent touches must be something they would proudly forward to a client.

The adoption win: agent screenshots a Shelter Prep scope summary and sends it on.

Design every deliverable for that moment.

PART 10

Success Metrics & The Floodgate

Metrics That Earn Phase 2

Phase 1 succeeds on signal quality, not vanity volume. Two agents using the workflow voluntarily and repeatedly is a stronger signal than fifty who tried it once.

METRIC	WHY IT MATTERS	PHASE 1 BAR
Repeat agent usage	The single most important signal — voluntary return without chasing.	**Majority of cohort returns unprompted**
Scope without rework	Proves interpretation is reliable; protects contractor trust.	**Clear majority of jobs**
Contractor response rate	Proves structured scopes are worth a contractor's time.	**Consistent, timely replies**
Vetted contractor count	Supply that lets demand ramp; seed of the moat.	**15–20 active**
Verified workflows	Volume of founder-verified jobs = seed operational memory.	**50+ verified**
Turnaround per inspection	The time-saved number agents feel and repeat to peers.	**Measurably faster than manual**

The Floodgate — The Single Most Important Gate in Phase 1

Phase 1's growth thesis is word of mouth. Velocity amplifies whatever it touches: if the loop is solid, referrals compound strength; if shaky, the same velocity compounds damage. Do not open beyond the 30-agent cohort until all five conditions below are true:

- ✓ **Inspection-to-scope interpretation is reliable — clean scope without the founder in the loop**
- ✓ **Contractor-ready scopes delivered without rework on a clear majority of jobs**
- ✓ **15–20 vetted contractors actively responding to structured scopes**
- ✓ **Documented repeat usage from cohort agents — they return unprompted**
- ✓ **Onboarding and vetting reduced to a transferable playbook a non-founder can run**

Every other Phase 1 decision is recoverable. Opening the floodgates before the loop is solid is the one that is not — because word of mouth cannot be un-spread.

The 30-Agent Cohort — Metered Waves

Wave 1	**5–8 agents live**	First vetted contractors onboarded; founder runs every job closely.
Wave 2	**+10 agents live**	Contractor pool at ~10; loop reliability proven on Wave 1.
Wave 3	**Remaining to 30 live**	Contractor pool at 15–20; scope delivered without rework consistently.

PART 11

The Complete Phased Prompt Sequence

One copy-paste prompt per step. Run them in order. Do not run the next until the previous passes its gate and you explicitly tell Codex to proceed. The Job Zero prompt is in Part 2; Steps 2–10 follow.

Step 2 · Make the Build Pass

```
Implement Step 2 only: make the build pass.
Do not add features. Do not touch schema. Do not add agents.
Fix only build/type/import/runtime issues created or exposed by Job Zero.
Run npm install, npm run build, npx tsc --noEmit.
STOP and report results + any risks. Do not proceed to Step 3.
```

Step 3 · Supabase Tables (migration files only)

```
Implement Step 3 only: canonical Supabase tables.
GENERATE migration files only. Do NOT apply them.
Tables: properties, work_requests, files, repair_items,
contractor_assignments, review_events, workflow_events.
files has NO signed_url column. workflow_events is immutable/insert-only.
STOP and report: migration summary, tables created, a reversible
down-migration, risk notes, and a manual test checklist.
Do not apply migrations. Do not proceed to Step 4.
```

Step 4 · File / Photo Storage

```
Implement Step 4 only: real file storage.
Supabase Storage private buckets. Permanent file records in the files table.
Signed URLs generated on demand server-side — never stored as a column.
Log signed-URL lifecycle and retrieval in file_access_events.
Do not add AI. STOP and report. Do not proceed to Step 5.
```

Step 5 · Auth, Roles, RLS (migration files only)

```
Implement Step 5 only: auth, roles, and RLS.
Roles: admin, agent/PM, contractor, viewer/seller. No hardcoded admin PIN.
GENERATE the RLS migration. Do NOT apply it.
STOP and report: policies created, a reversible down-migration,
risk notes, and a manual test checklist for cross-tenant access.
Do not proceed to Step 6.
```

Step 6 · Server-Side Trust Spine

```
Implement Step 6 only: server-side trust spine.
Enforce workflow transitions, review_required, human_verified, audit logs,
immutable workflow_events, failure records, idempotency — all server-side.
Browser must not decide verification. Confidence is triage only.
STOP and report. Do not proceed to Step 7.
```

Step 7 · Work-Request / File Loop UI

```
Implement Step 7 only: the operational loop UI.
Primary statuses: New, In Progress, Ready, Done.
Secondary flags as listed in Part 9.
Keep it calm. No heavy construction-software surface area.
STOP and report. Do not proceed to Step 8.
```

Step 8 · Manual Repair Item Review (what is wrong)

```
Implement Step 8 only: manual repair item review.
Tables: repair_items, missing_information, admin_corrections, review_events.
Manual approve/edit/reject flow. Founder creates and corrects by hand.
Generate any migration files only; STOP and report before apply.
Do not proceed to Step 9.
```

Step 9 · Job Execution Scope Structures (how work gets done)

```
Implement Step 9 only: job execution scope structures.
Tables: job_execution_steps, scope_notes, estimate_review_owner.
Fields for materials/tools/equipment, access, safety, cleanup/disposal.
Generate any migration files only; STOP and report before apply.
Do not proceed to Step 10.
```

Step 10 · Six Agents — one at a time

```
Implement Step 10: enable ONE agent only, named in this prompt.
Allowed agents: Intake, Inspector, Gap Detection, Scope Drafter, Report,
UX/Presentation. Estimate and Routing stay manual.
Do NOT scaffold the future taxonomy or any deferred agent.
STOP and report after each single agent. Repeat this prompt per agent.
```

APPENDIX

Canon Checklist & v2 Change Log

Required Repo Files — Lock the Canon

Ensure these exist before Codex begins any build step:

```
/AGENTS.md
/docs/README.md
/docs/phase-1-executable-spec.md
/docs/reconciliation-map.md
/docs/grand-master-plan.md
/docs/ai-orchestration-v2.md
/docs/review-gate-doctrine.md
/docs/JOB-ZERO-PROMPT.md
```

v2 Change Log — Four Fixes Applied

1 · Phased execution	Added the execution protocol and stop-and-report template (Part 1). The PDF is the locked authority document; Codex runs one step per pass, Job Zero first. Full per-step prompts added in Part 11.
2 · files schema	Removed signed_url_expiry from files. File records are permanent (id, bucket, path, original_name, mime, size, uploaded_by, created_at). Signed URLs generated on demand; lifecycle tracked in file_access_events.
3 · Step 8 / 9 split	Step 8 = Manual Repair Item Review (what is wrong: repair_items, missing_information, admin_corrections, review_events). Step 9 = Job Execution Scope Structures (how work gets done: job_execution_steps, scope_notes, materials/tools, access/safety/cleanup, estimate_review_owner).
4 · No silent migrations	Database safety rule added (Part 1). Codex generates migration files but never assumes they are applied — it stops and reports summary, down-migration, risk notes, and a manual test checklist. A human applies.

What Codex Must Never Build in Phase 1

> **DO NOT build: Seller Prep portal · Predictive maintenance · Full contractor marketplace · Pricing automation · Source research agents · Observer/Analyst agent · Estimate agent (deferred to Phase 2) · Routing agent (deferred to Phase 2)**

Phase Transition Trigger to Phase 2

Phase 2 begins when repeat agent usage, contractor response consistency, and stable workflow routing all hold together — and enough founder-verified jobs exist that the Estimate agent has real pricing memory to retrieve from. The metric opens the gate, not the calendar.

"Approve it. Run Job Zero only. Hold the floodgate until the interpretation is undeniable."
